Customer Complaint Classification

Author: Samira N Ray **Course:** Udacity Azure Generative AI Engineer Nanodegree, Project 3 **Date:** October 13, 2026

Repository: https://github.com/Rei0322/azure-complaint-classifier

1. Overview

This project builds an end-to-end generative AI workflow on Azure OpenAI that turns a customer's recorded audio complaint into a classified support case with visual evidence. Each complaint is transcribed, turned into an image prompt, rendered as an image of the reported problem, described and annotated by a vision model, and finally classified into a category and subcategory from the provided product catalog. Every intermediate result is saved so each stage can be reviewed.

2. Architecture

Step	Module	Model / tool	Output
1. Transcribe audio	whisper.py	Whisper	transcription.txt
2. Build image prompt	dalle.py	Prompt template	image_prompt.txt
3. Generate image	dalle.py	gpt-image-1-mini	generated_image.png
4. Describe image	vision.py	GPT-4o (vision), JSON output	image_description.txt
5. Annotate issue	vision.py	Pillow (bounding box from step 4)	annotated_image.png
6. Classify complaint	gpt.py	GPT-4o, JSON mode + catalog check	classification.txt

All three models are deployed in one Azure AI Foundry resource in East US 2 (Global Standard / Standard, under the lab's 10K TPM policy). Settings load from a local .env file so the API key is never committed.

3. Results

	Customer 1	Customer 2
Reported issue	Torn material and significant damage to the front of the shoe.	Torn material on the front of the shoe
Category	Fashion	Fashion
Subcategory	Shoes & Accessories	Shoes & Accessories
Reason	The complaint is about the poor quality and damage of a recently purchased pair of shoes.	The complaint is about the quality and damage of a pair of shoes, which falls under the 'Shoes & Accessories' subcategory in the 'Fashion' category.

Both complaints describe new shoes with torn material at the front, and both were correctly classified as **Fashion > Shoes & Accessories**. The vision model's bounding box correctly placed the annotation on the damaged toe area in both generated images.

Generated and annotated images

Customer 1



Left: generated image. Right: annotated image with the reported issue boxed.

Customer 2



Left: generated image. Right: annotated image with the reported issue boxed.

Transcription (Customer 1)

Hi, I am contacting you regarding a pair of shoes I recently purchased. Unfortunately, the quality is far from what I expected. As you can see, the shoe is completely ruined. The material is torn, and there is significant damage to the front. This is not something that should happen with a new pair of shoes, and I am extremely disappointed. I am requesting a refund or a replacement as soon as possible. Please let me know how to proceed with the return process. Thank you for your attention to this matter.

4. Design decisions

Image model. DALL-E 3 was not available to deploy, so I used gpt-image-1-mini, its current replacement. The image function handles both URL and base64 responses, so either model works.

Structured outputs. The vision and classification steps return JSON. That lets the bounding box drive the annotation directly, and the classifier's answer is checked against categories.json so an invented category gets flagged.

Prompting. The image prompt asks for a realistic product photo focused on the defect, with no text or faces, which gives the vision step a clear issue to find.

Security and cost. Credentials live in .env (git-ignored). The script can run a single complaint (python main.py customer1) to keep testing within the lab's $25 budget.

5. Challenges and lessons learned

Region availability. Whisper and the image models are offered in only a few regions, so East US 2 was chosen because it hosts all three models.

Deployment policy. The lab enforces a 10K tokens-per-minute cap, so deployments had to be sized down to pass policy.

Endpoint confusion. The Foundry project URL (services.ai.azure.com/api/projects/...) returned 'API version not supported'. Switching to the resource's Azure OpenAI endpoint (openai.azure.com) with API version 2025-04-01-preview fixed it.

Annotation readability. The first labels were too small to read. I resized them relative to the image and shrank long labels so they always fit.

6. Possible improvements

Add a simple web UI where users upload a recording and watch each step run, batch-process many complaints with a cost cap, and send the classification straight into a ticketing system with the annotated image attached as evidence.